Lesson 2 Teacher's Key

Godot Engine

INSTRUCTOR ONLY - do not distribute to students

Level 1 - Complete Worked Solution

Attach to Player's Sprite2D root - works the same whether you're editing player.tscn directly or selecting Player inside main.tscn:

```
func _process(delta):
    if Input.is_action_pressed("ui_right"):
        position.x += 200 * delta
```

Level 2 - Complete Worked Solution

Attach this script to the Sprite2D root. Using Input.get_vector() is the idiomatic Godot way to combine 4 directions into one movement call:

```
extends Sprite2D

var speed := 200

func _process(delta):
    var dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    position += dir * speed * delta
```

Level 3 - All Worked Solutions

Every stretch-goal idea from the handout, fully worked - not just one. Useful for validating each idea actually works before class, and as a ready reference no matter which one a student ends up picking (or if a student blows past all of them, this is your own jumping-off point for exploring further right alongside them).

Animate: blink while moving, static when idle

The simplest blink in Godot is toggling the built-in visible property (inherited from CanvasItem) on and off based on an accumulating timer - no AnimationPlayer or sprite swap needed:

```
var blink_time := 0.0

func _process(delta):
    var dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    position += dir * speed * delta
    if dir != Vector2.ZERO:
        blink_time += delta
        visible = int(blink_time * 6) % 2 == 0
    else:
```

```
        visible = true
        blink_time = 0.0
```

Sound: beep-beep while moving, silent when idle

Add an AudioStreamPlayer2D node as a child, set its Stream to the imported sfx_blip.wav. Checking .playing before calling .play() again means it naturally retriggers as soon as the short clip finishes - a beep-beep rhythm, not one continuous tone. Calling .stop() when idle is what makes it actually silent, not just "not retriggered":

```
    if dir != Vector2.ZERO:
        if not $AudioStreamPlayer2D.playing:
            $AudioStreamPlayer2D.play()
    else:
        $AudioStreamPlayer2D.stop()
```

Recolor: different sprite per direction, reset when idle

Same Idle/Moving concept as the State Diagram from this lesson's bonus section. This swaps the texture itself rather than tinting it with modulate - preload the other 4 placeholder circles alongside your default one, then assign texture directly:

```
const TEX_DEFAULT = preload("res://player_circle_orange.png")
const TEX_RIGHT = preload("res://player_circle_red.png")
const TEX_LEFT = preload("res://player_circle_blue.png")
const TEX_UP = preload("res://player_circle_green.png")
const TEX_DOWN = preload("res://player_circle_yellow.png")

    if dir == Vector2.ZERO:
        texture = TEX_DEFAULT        # idle
    elif dir.x > 0:
        texture = TEX_RIGHT
    elif dir.x < 0:
        texture = TEX_LEFT
    elif dir.y < 0:
        texture = TEX_UP
    else:
        texture = TEX_DOWN
```

Clamp movement to the screen

clamp() on position.x/y is the standard Godot boundary pattern; get_viewport_rect().size (inherited from CanvasItem) gives the current screen size dynamically rather than hardcoding numbers:

```
    var vp := get_viewport_rect().size
    position.x = clamp(position.x, 0, vp.x)
    position.y = clamp(position.y, 0, vp.y)
```

Structure: group everything triggered by the same action in one place

Godot's _process(delta) already runs once per frame, so the "same place in code" for this lesson is that one function - the trick is organizing what goes inside it, not spreading logic across multiple functions or nodes. Movement and the screen clamp always run, every frame, regardless of direction. Blink, sound, and recolor are all direction-dependent, so they all live under one shared if dir != Vector2.ZERO / else check - not four separate direction checks scattered through the function. That's the Complete Level 3 solution below.

Complete Level 3 solution (all 5 goals combined)

Everything above combined into one script, attached to the Sprite2D root (with an AudioStreamPlayer2D child node, Stream set to sfx_blip.wav):

```
extends Sprite2D

const TEX_DEFAULT = preload("res://player_circle_orange.png")
const TEX_RIGHT = preload("res://player_circle_red.png")
const TEX_LEFT = preload("res://player_circle_blue.png")
const TEX_UP = preload("res://player_circle_green.png")
const TEX_DOWN = preload("res://player_circle_yellow.png")

var speed := 200.0
var blink_time := 0.0

func _process(delta):
    var dir := Input.get_vector("ui_left", "ui_right", "ui_up", "ui_down")
    position += dir * speed * delta

    var vp := get_viewport_rect().size
    position.x = clamp(position.x, 0, vp.x)
    position.y = clamp(position.y, 0, vp.y)

    if dir != Vector2.ZERO:
        blink_time += delta
        visible = int(blink_time * 6) % 2 == 0
        if not $AudioStreamPlayer2D.playing:
            $AudioStreamPlayer2D.play()
        if dir.x > 0:
            texture = TEX_RIGHT
        elif dir.x < 0:
            texture = TEX_LEFT
        elif dir.y < 0:
            texture = TEX_UP
        else:
            texture = TEX_DOWN
    else:
        visible = true
        blink_time = 0.0
        $AudioStreamPlayer2D.stop()
        texture = TEX_DEFAULT
```